Workgroup: Network Working Group
Internet-Draft: draft-wullink-rpp-core-05
Published: 15 March 2026
Intended Status: Standards Track
Expires: 16 September 2026
Authors: M. Wullink, *SIDN Labs*; P. Kowalik, *DENIC*

RESTful Provisioning Protocol (RPP)

Abstract

This document describes the endpoints for the RESTful Provisioning Protocol, used for the provisioning and management of objects in a shared database.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 16 September 2026.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 5
2. Terminology 5
3. Conventions Used in This Document 5
4. Request Headers 6
5. Response Headers 7
6. Error handling and relation between HTTP status codes and RPP codes 7
7. Problem Detail responses for errors 9
8. Bootstrapping 10
9. Discoverability 11
9.1. Workflow 13
10. Versioning 14
10.1. Endpoints 14
10.2. Messages 14
10.3. Extensions 15
10.4. Profiles 15
11. Media types 15
12. Profiles 16
12.1. Definition 16
12.2. Inheritance 17
12.3. Signalling 18
12.3.1. Header signalling 18
12.3.2. Media type parameter signalling 19
13. Endpoints 19
13.1. Availability for Creation 20
13.2. Resource Information 21
13.3. Poll for Messages 22
13.4. Delete Message 22
13.5. Create Resource 23

13.6. Delete Resource 24
13.7. Processes Path Segment 25
13.7.1. Generic proces interface 25
13.7.1.1. Starting: 25
13.7.1.2. Cancelling: 26
13.7.1.3. Status 26
13.7.1.4. Other operations 26
13.7.1.5. Listing 27
13.7.2. Relation to object representation 27
13.8. Renew Resource 28
13.9. Transfer Resource 29
13.9.1. Start 29
13.9.2. Status 31
13.9.3. Cancel 32
13.9.4. Reject 33
13.9.5. Approve 33
13.10. Update Resource 34
14. Extension Framework 35
15. RPP Result Codes 35
16. Authentication and Authorization 36
16.1. OAuth 2.0 36
16.2. Scopes 37
16.2.1. Scope Derivation Rules 37
16.2.2. Scope Registry 38
16.2.3. Rich Authorization Requests (RAR) 39
16.3. Claims 40
16.3.1. Required Claims 40
16.3.2. RPP-Specific Claims 41
16.3.3. Claim Validation 44

16.4. Flows 45
16.4.1. Machine to Machine 45
16.4.2. Interactive 47
16.4.2.1. Registrar Employee Flow 47
16.4.2.2. Registrant Customer Flow 48
16.4.3. Enforcing Interactive Authentication for High-Risk Operations 50
16.5. Secure Object Transfer 50
16.5.1. Federation Trust Model 52
16.5.2. Interactive Flow 54
16.5.3. Fallback Flow 57
17. IANA Considerations 59
17.1. URN Sub-namespace for RPP (urn:ietf:params:rpp) 59
17.2. RPP registry group 59
17.3. RPP Discovery registry 59
17.4. RPP Extension registry 60
17.5. RPP Profile registry 60
17.6. RPP Result Codes Registry 61
17.7. RPP Media Type (application/rpp+json) 61
18. Internationalization Considerations 61
19. Security Considerations 62
20. Change History 62
20.1. Version 05 to 06 62
20.2. Version 04 to 05 62
20.3. Version 03 to 04 62
20.4. Version 02 to 03 62
20.5. Version 01 to 02 62
20.6. Version 00 to 01 63
20.7. Version 00 (draft-rpp-core) to 00 (draft-wullink-rpp-core) 63
21. Acknowledgements 63

22. References 63
22.1. Normative References 63
22.2. Informative References 65
Authors' Addresses 66

1. Introduction

This document describes an Application Programming Interface (API) API based on the HTTP protocol [RFC2616] and the principles of [REST]. Conforming to the REST constraints is generally referred to as being "RESTful". Hence the API is dubbed: "'RESTful Provisioning Protocol" or "RPP" for short.

The RPP API is designed to be used for the provisioning and management of objects in a shared database, such as domain names, hosts, and entities.

2. Terminology

In this document the following terminology is used.

REST - Representational State Transfer ([REST]). An architectural style.

RESTful - A RESTful web service is a web service or API implemented using HTTP and the principles of [REST].

EPP RFCs - This is a reference to the EPP version 1.0 specifications [RFC5730], [RFC5731], [RFC5732] and [RFC5733].

RESTful Provisioning Protocol or RPP - The protocol described in this document.

URL - A Uniform Resource Locator as defined in [RFC3986].

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

RPP client - An HTTP user agent performing an RPP request

RPP server - An HTTP server responsible for processing requests and returning results in any supported media type.

JWT - JSON Web Token as defined in [RFC7519].

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT","SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

In examples, indentation and white space in examples are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

All example requests assume a RPP server using HTTP version 2 is listening on the standard HTTPS port on host rpp.example. An authorization token has been provided by an out of band process and MUST be used by the client to authenticate each request.

4. Request Headers

A RPP request does not always require a request message body. The information conveyed by the HTTP method, URL, and request headers may be sufficient for the server to be able to successfully processes a request. However, the client MUST include a request message body when the server requires additional attributes to be present in the request message. The RPP HTTP headers listed below use the "RPP-" prefix, following the recommendations in [RFC6648].

- RPP-Cltrid: The client transaction identifier is the equivalent of the clTRID element defined in [RFC5730] and MUST be used accordingly, when the HTTP message body does not contain an EPP request that includes a cltrid.
- RPP-Authorization: The client MAY use this header to send authorization information in the format <method> <authorization information>, similar to the HTTP Authorization header, defined in [RFC9110, Section 11.6.2]. The <method> indicates the type of authorization being used. For EPP object authorization information, for example the authorization information used for domain names described in [RFC5731, Section 2.3], a new authinfo method is defined and MUST be used. The <authorization information> defines the following comma separated fields:
  - value (REQUIRED): Base64 encoded EPP password-based authorization information. Base64 encoding is used to prevent problems when special characters are present that may conflict with the format rules for the Authorization header.
  - roid (OPTIONAL): A Roid as defined in [RFC5731], [RFC5733], and [RFC5730]. The roid is used to identify the object for which the authorization information is provided. If the roid is not provided, then the server MUST assume that the authorization information is linked to the object identified by the URL of the request.

Use of the RPP-Authorization header:

```
RPP-Authorization: authinfo value=TXkgU2VjcmFRva2Vu, roid=REG-X-123
```

The value of the RPP-Authorization header is case sensitive. The server MUST reject requests where the case of the header value does not match the expected case. The RPP-Authorization header is specific to the user agent and MUST NOT be cached, as recommended by Section 16.4.2, the server MUST use the correct HTTP cache directives to prevent caching of the RPP-Authorization header.

- RPP-Profile: The client MUST use this header to indicate the profiles is used in the request.

5. Response Headers

The server HTTP response contains a status code, headers, and MAY contain an RPP response message in the message body. HTTP headers are used to transmit additional data to the client and MAY be used to send RPP process related data to the client. HTTP headers used by RPP MUST use the "RPP-" prefix, the following response headers have been defined for RPP.

- `RPP-Svtrid`: This header is the equivalent of the "svTRID" element defined in [RFC5730] and MUST be used accordingly when the RPP response does not contain an EPP response in the HTTP message body. If an HTTP message body with the EPP XML equivalent "svTRID" exists, both values MUST be consistent.
- `RPP-Cltrid`: This header is the equivalent of the "clTRID" element defined in [RFC5730] and MUST be used accordingly when the RPP response does not contain an EPP response in the HTTP message body. If the contents of the HTTP message body contains a "clTRID" value, then both values MUST be consistent.
- `RPP-Code`: This header is the equivalent of the EPP result code defined in [RFC5730] and MUST be used accordingly. This header MUST be added to all responses and MAY be used by the client for easy access to the result code, without having to parse the HTTP response message body.

For the EPP codes related to session management (1500, 2500, 2501 and 2502) there are no corresponding RPP codes.

In order for RPP to be backwards compatible with EPP, RPP will use 5-digit coding of the result codes, where first digit will denote origin specification of the result codes.

For [RFC5730] Result Codes the leading digit MUST be "0". For RPP result codes the leading digit MUST be "1". For avoidance of confusion RPP MUST not define new codes with the same semantic meaning as already defined in EPP.

For RPP codes the remaining 4 digits MUST keep the same semantics as [RFC5730] Result Codes.

- `RPP-Queue-Size`: Return the number of unacknowledged messages in the client message queue. The server MAY include this header in all RPP responses.

6. Error handling and relation between HTTP status codes and RPP codes

RPP leverages standard HTTP status codes to reflect the outcome of RPP operations. The RPP result codes are based on the EPP result codes defined in [RFC5730]. This allows clients to handle responses generically using common HTTP patterns. While the HTTP status code provides the primary, high-level outcome, the specific RPP result code MUST still be provided in the `RPP-Code` HTTP header for detailed diagnostics.

The mapping strategy is to use the most specific HTTP code that accurately reflects the operation's result.

For common and well-defined outcomes, a specific HTTP status code is used. For example, an attempt to access a non-existent resource (EPP code 2302) MUST return 404 Not Found, and an attempt to create a resource that already exists (EPP code 2303) MUST return 409 Conflict. This allows a client to handle these common situations based on the HTTP code alone.

For all other failures, a generic HTTP status code is used. Client-side errors (e.g., syntax, parameter, or policy violations) MUST return 400 Bad Request. Server-side failures MUST return 500 Internal Server Error.

The server MUST return HTTP status codes, following the mapping rules in Table 1.

Table 1: RPP result code and HTTP Status-Code mapping.

HTTP Status-Code	Description	Corresponding RPP result code(s)
Success (2xx)		
200 OK	The request was successful (e.g., for GET or UPDATE).	01000 (in all cases not specified otherwise), 01300, 01301
201 Created	The resource was created successfully.	01000 for resource creating requests (POST/PUT)
202 Accepted	The request was accepted for asynchronous processing.	01001
204 No Content	The resource was deleted successfully.	01000 for DELETE
Client Errors (4xx)		
400 Bad Request	Generic client-side error (syntax, parameters, policy).	02000-02005, 02104-02106, 02300-02301, 02304-02308
403 Forbidden	Authentication or authorization failed.	02200-02202
404 Not Found	The requested resource does not exist.	02303
409 Conflict	The resource could not be created because it already exists.	02302
Server Errors (5xx)		
500 Internal Server Error	Generic server-side error; command failed.	02400

HTTP Status-Code	Description	Corresponding RPP result code(s)
501 Not Implemented	The requested command or feature is not implemented.	02100-02103

Table 1

Some EPP result codes, like 01500, 02500, 02501 and 02502 are related to session management and therefore not applicable to a sessionless RPP protocol.

7. Problem Detail responses for errors

When an error occurs that prevents processing of the requested action, an RPP server MUST respond using a Problem Detail document [RFC9457] detailing what went wrong, or what was not acceptable to the server. The `type` field MUST be the `urn:ietf:params:rpp:problem` URN. The `status` field MUST reflect the HTTP status code. The document MUST contain an `errors` element, as a list of objects detailing individual errors.

This document consists of the following fields:

`type` (required, string) This field MUST always contain the fixed string value `urn:ietf:params:rpp:error` to indicate that this is an RPP Problem Detail response.
`title` (required, string) A short human-readable description of the Problem Detail type.
`errors` (optional, array of error objects) MUST contain objects detailing information about the specific errors that occurred, including optional references to which values in the original request were not acceptable to the server.

The `error` object consist of the following fields:

`type` (required, string) This field SHOULD be a URI that identifies the error type. This URI MAY be dereferenceable to obtain human-readable documentation about the error type.
`result` (required, string) This field MUST contain the RPP result code associated with the error.
`paths` (optional, list of strings) The JSONPath [RFC9535] to the value(s) in the request that caused the error. This field MAY be omitted if no specific value in the request can be identified as the cause of the error.
`reason` (required, string) A human-readable detailed description of the error.

RPP specifications and/or extensions MAY add extension fields to the error object, to convey additional information about the causes of the error. For example, to indicate the account balance on a billing failure, the following Problem Detail response could be used:

```
{
 "type": "urn:ietf:params:rpp:error",
 "title": "RPP Error",
 "errors": [{
  "type": "https://rpp.example/problems/billing/billing-failure",
  "result": "02104",
  "reason": "Not enough balance on account to create domain",
  "balance": 10.0,
  "action_cost": 25.0
 }]
}
```

Problem Detail response containing multiple errors for a domain create request using an invalid domain name (_$.example), JSONPaths are specified in the error objects to indicate which value in the request caused the error:

```
{
 "type": "urn:ietf:params:rpp:error",
 "title": "RPP Error",
 "errors": [{
  "type": "https://rpp.example/problems/domains/domain-syntax",
  "result": "02005",
  "paths": ["$.domain.name"],
  "reason": "Invalid character(s) in domain name",
 },
 {
  "type": "https://rpp.example/problems/domains/domain-allowed-length",
  "result": "02004",
  "paths": ["$.domain.name"],
  "reason": "Domain name length must be between 3 and 63 characters",
 }]
}
```

8. Bootstrapping

The client MUST be able to bootstrap itself by discovering the location of an RPP server. Not having a fixed location for the RPP server is a fundamental design principle of RPP, as it allows for a more flexible and scalable architecture. The client MUST use either the IANA registry for RPP servers or a DNS lookup using an SRV record as defined in [RFC2782]. The format and procedure for adding an RPP server to the IANA registry is defined in the IANA Considerations section below.

For DNS-based bootstrapping, an RPP server MUST publish an SRV record in each DNS zone that is served by the RPP server. The owner name of the SRV record MUST be `_rpp._tcp.<zone>`.

If multiple SRV records are returned, the client MUST select the RPP server according to the priority and weight rules in [RFC2782]. The client MUST ignore SRV records with a target of `.` (service not available).

The SRV record provides the target host and port of the RPP service. The client MUST construct the URL for the well-known endpoint (defined in the Discoverability section below) as:

- https://<target>:<port>/.well-known/rpp when <port> is not 443
- https://<target>/.well-known/rpp when <port> is 443

The client MUST use the constructed URL to discover the capabilities of the RPP server, as defined in the Discoverability section below.

Example SRV record for an RPP server for the TLD "example" running HTTPS on port 443 at rpp.example.:

```
_rpp._tcp.example. 3600 IN SRV 0 0 443 rpp.example.
```

In this example, the well-known endpoint URL is https://rpp.example/.well-known/rpp.

9. Discoverability

RPP server capabilities MUST be discoverable by clients. The server MUST provide a well-known endpoint at /.well-known/rpp at the root of the RPP server, this endpoint MUST return a JSON document containing the capabilities of the RPP server. The well-known endpoint MUST be accessible without authentication, and the client MUST be able to access this endpoint before authenticating with the server. The well-known endpoint MUST be accessible using the HTTP GET method and MUST return an HTTP status code 200 (OK) if the request was successful. The response message body MUST contain a JSON document describing the capabilities of the RPP server using the following fields:

- base_url: (required, string) The base URL for the RPP API, this is the URL that MUST be used as the base for all endpoint URL templates.
- version: (required, string) The version of the RPP API supported by the server, for example "1.0".
- tlds: (required, array of strings) A list of TLDs supported by the server, for example "example", "org".
- extensions: (optional, array of extension objects) A list of supported extensions, each extension object MUST contain the following fields:
  - name: (required, string) A short name for the extension, for example "registry fee extension".
  - id: (required, string) A unique URN identifier for the extension, for example "urn:ietf:params:rpp:extension:registry-fee".
  - version: (required, string) The version of the extension supported by the server, for example "1.0".
  - url: (required, string) for standard extensions, this MUST be the URL for the extension in the IANA registry, for private extensions, this MUST be the URL for the extension specification, the domain name used in the URL MUST resolve and the URL MUST be accessible to the client.

- profiles: (optional, array of profile objects) A list of supported profiles, each profile object MUST contain the following fields:
  - name: (required, string) A short name for the profile, for example "domain provisioning profile".
  - id: (required, string) A unique URN identifier for the profile, for example "urn:ietf:params:rpp:profile:domain-provisioning-1.0".
  - version: (required, string) The version of the profile supported by the server, for example "1.0".
  - url: (required, string) for standard profiles, this MUST be the URL for the profile in the IANA registry, for private profiles, this MUST be the URL for the profile specification, the domain name used in the URL MUST resolve and the URL MUST be accessible to the client.
- objects: (required, array) A list of supported resource collections, for example "domains", "hosts", "entities".
- authentication: (optional, array of strings) A list of supported authentication methods, for example "Bearer", "Basic".
- endpoints: (required, array of endpoint objects) A list of available endpoints, each endpoint object MUST contain the following fields:
  - name: (required, string) A short name for the endpoint, for example "availability", "info", "poll", "create", "delete", "renewal" or "transfer".
  - url_template: (required, string) The URI template for the endpoint, using the syntax defined in [RFC6570].
- maintenance: (optional, array) An array containing information about upcoming planned maintenance windows of the server, with the following fields:
  - start_time: (required, string) The start time of the maintenance window in ISO 8601 format.
  - end_time: (required, string) The end time of the maintenance window in ISO 8601 format.
  - description: (optional, string) A human-readable description of the maintenance window.

The following template variables are defined for use in RPP endpoint URL templates:

- collection: The resource collection type (e.g., "domains", "hosts", "entities")
- id: The unique identifier for a resource instance within a collection
- process_name: The name of a process associated with a resource (e.g., "transfers", "renewals")
- process_id: The unique identifier for a specific process instance

Example discovery response document:

```
{
 "base_url": "https://rpp.example/rpp/v1",
 "version": "1.0",
 "tlds": ["example", "org"],
 "extensions": [
   {
    "name": "RPP example extension",
```

```
      "id": "urn:ietf:params:rpp:extension:example-extension",
      "version": "1.0",
      "url": "https://www.iana.org/assignments/rpp-extensions/rpp-example-extension-1.0"
    }
  ],
  "profiles": [
    {
      "name": "EPP compatibility profile",
      "id": "urn:ietf:params:rpp:profile:epp-compatibility",
      "version": "1.0",
      "url": "https://www.iana.org/assignments/rpp-profiles/epp-compatibility-provisioning-
profile-1.0"
    }
  ],
  "objects": ["domains", "hosts", "entities"],
  "endpoints": [
    {
      "name": "availability",
      "url_template": "/{collection}/{id}/availability"
    },
    {
      "name": "info",
      "url_template": "/{collection}/{id}"
    },
    {
      "name": "poll",
      "url_template": "/messages"
    },
    {
      "name": "create",
      "url_template": "/{collection}"
    },
  ],
  "authentication": ["Bearer"],
  "maintenance": [
    {
      "start_time": "2026-06-01T00:00:00Z",
      "end_time": "2026-06-01T06:00:00Z",
      "description": "Planned maintenance for server upgrades"
    }
  ]

}
```


9.1. Workflow

The steps for a typical workflow of provisioning an object using RPP without knowing the location and capabilities of the server are as follows, the first three steps are optional, the client can choose to skip any of these steps if it already has the required information from a previous interaction or configuration.

1. Bootstrap (optional): The client discovers the location of the RPP server by looking up the IANA registry for RPP servers or by performing a DNS SRV lookup as defined in [RFC2782].

2. Discover capabilities (optional): The client retrieves the capabilities of the RPP server by sending a GET request to the well-known endpoint at /.well-known/rpp.
3. Extract RPP URLs (optional): The client extracts the base URL and endpoint URL templates from the discovery response, and uses this information to construct the URLs for the desired operations.
4. Perform provisioning operations: The client performs provisioning operations by sending HTTP requests to the appropriate endpoint URLs, using the HTTP method and request message body as required by the specific operation.

10. Versioning

RPP is designed to be extensible and backward compatible. The version of the RPP API is indicated in the URL path, for example: https://rpp.example/rpp/v1/. The server MUST support at least one version of the RPP API, and MUST return a 404 Not Found status code for requests using an unsupported version. The versioning scheme uses the Semantic Versioning format defined in [SemVer], but only the major version number is used to indicate breaking changes. The minor and patch version numbers are not used in an URL path, but can be used in the media type or in the message body to indicate non-breaking changes.

The following RPP elements include versioning support:

- Endpoints: The server MUST support at least one version of the RPP API, and MUST return a 404 Not Found status code for requests using an unsupported version.
- Messages: A request and response message MUST include the version of the RPP API it is compatible with.
- Extensions: RPP extensions MUST include the version of the RPP API they are compatible with.
- Profiles: RPP profiles MUST include the version of the RPP API they are compatible with.
- Media types: RPP media types MUST include the version of the RPP API they are compatible with.
- Result codes: RPP result codes may be added by extensions or updates to the core RPP specification.

10.1. Endpoints

The `base_url` element of the RPP Discovery response MAY include the version of the RPP API supported by the server. The client MUST use this `base_url` for all subsequent requests to the server. For example, if the version is 1.2.3, the `base_url` is https://rpp.example/rpp/v1/, then the client MUST use this URL for all subsequent requests to the server, and MUST not use a different version in the URL path.

10.2. Messages

The `version` element of the RPP request and response messages MUST include the version of the RPP API that the message is compatible with. The server MUST reject requests with a version that is not supported by the server, and MUST return a RPP Client error code.

10.3. Extensions

The RPP server MUST include the version for each extension in the RPP Discovery response. The client MUST use this version information to determine which extensions are supported by the server, and to ensure that it uses the correct version of the extension when making requests to the server. A request using an extension MUST include the version of the extension. The server MUST reject requests using extensions with a version that is not supported by the server, and MUST return a RPP Client error code. The following is an example of how the version information for an extension can be included in the RPP Discovery response:

```
"extensions": [
   {
    "name": "RPP example extension",
    "id": "urn:ietf:params:rpp:extension:example-extension",
    "version": "1.0",
    "url": "https://www.iana.org/assignments/rpp-extensions/rpp-example-extension-1.0"
   }
 ],
```

10.4. Profiles

The RPP server MUST include the version for each profile in the RPP Discovery response. The client MUST use this version information to determine which profiles are supported by the server, and to ensure that it uses the correct version of the profile when making requests to the server. A request using a profile MUST include the version of the profile. The server MUST reject requests using profiles with a version that is not supported by the server, and MUST return a RPP Client error code. The following is an example of how the version information for a profile can be included in the RPP Discovery response:

```
"profiles": [
   {
    "name": "EPP compatibility profile",
    "id": "urn:ietf:params:rpp:profile:epp-compatibility",
    "version": "1.0",
    "url": "https://www.iana.org/assignments/rpp-profiles/epp-compatibility-provisioning-
profile-1.0"
   }
 ]
```

11. Media types

RPP media types are used to indicate the format of the request and response messages, and MUST include a parameter indicating the name of the profile they are compatible with. The server MUST use the profile information in the media type to determine which features, extensions and versions to use when processing the request, and to ensure that it

returns a response that is compatible with the client. The client MUST use the profile information in the media type to determine which features, extensions and versions to use when processing the response, and to ensure that it can correctly interpret the response.

The definition of profile parameters in media types is described in section ....

12. Profiles

A profile is a named set of protocol features and versions that are used to define the compatibility and capabilities of RPP server implementations, allowing for better interoperability between different implementations. Using profiles helps to simplify the implementation and deployment of RPP by providing a clear and concise way for the client and server to communicate their capabilities and requirements.

A profile is identified by a unique name and may be published as a standard profile in the IANA registry for RPP profiles, to promote interoperability and standardization across implementations. Standard profiles names MUST use the RPP URN namespace defined in this document, for example: `urn:ietf:params:rpp:profile:{profile_name}`. The profile name MUST be unique within the RPP URN namespace and SHOULD be descriptive of the features and versions included in the profile.

A profile can also be defined as a private profile, which is not published in the IANA registry, but is used by specific server implementations only. A private profile can be defined by a server operator to specify the features and versions supported by their implementation. If the profile is not published in the IANA registry, then the server operator MUST ensure that the profile name is globally unique to avoid conflicts with other profiles, the use of reverse domain name notation is RECOMMENDED for private profiles to ensure uniqueness.

12.1. Definition

A profile definition MUST contain the following fields, private profiles may contain additional fields as needed:

- `name`: A unique name that identifies the profile.
- `description`: A human-readable description of the profile and its intended use.
- `version`: The version of the profile.
- `rpp_version`: The minimum version of the RPP protocol that is supported or required for the profile.
- `objects`: A list of objects allowed for provisioning operations, for example "domains", "hosts", "entities", etc.
- `extensions`: A list of extensions, including their versions, that are supported or required for the profile.
- `profile`: A base profile that is extended by this profile. The base profile MUST be published in the IANA registry for RPP profiles, or be made available to the client using other means.

Example JSON representation for a standard profile named "example-profile" that supports RPP version 1.0 and includes two extensions, "rppExample" version 1.0 and "rppOther" version 1.1. The profile also "extends" the "base-profile" profile, which is defined in the IANA registry for RPP profiles and supports RPP version 1.0.

```
{
 "name": "urn:ietf:params:rpp:profile:example-profile",
 "description": "An example profile for provisioning objects using the RPP protocol.",
 "version": "1.0",
 "rpp_version": "1.0",
 "objects": ["domains", "hosts", "entities"],
 "extensions": [
   {
     "rppExample": {
       "version": "1.0"
     },
     "rppOther": {
       "version": "1.1"
     }
   }
 ],
 "profile": {
   "name": "urn:ietf:params:rpp:profile:base-profile",
   "version": "1.0"
 }
}
```

12.2. Inheritance

A profile can include another profile, which is referred to as "inheriting" from that profile. When a profile inherits from another profile, it means that the features and versions defined in the inherited profile are also included in the inheriting profile. This allows for the creation of more complex profiles by building upon existing profiles. For example, a profile named "example-profile" could inherit from a base profile named "base-profile", which includes a set of common features and versions. The "example-profile" could then add additional features and versions on top of the ones defined in the "base-profile", or it can override some of the features and versions from the "base-profile". This allows for greater flexibility and modularity in defining profiles, as well as promoting reuse of common features and versions across different profiles. However, it is important to note that the use of inheritance in profiles can also make things more complicated, as it can create dependencies between profiles and make it harder to understand the features and versions included in a profile. Therefore, it is recommended to use inheritance with caution and to clearly document the relationships between profiles. The depth of inheritance MUST be limited to 1 to provent excessive complexity, and profile designers MUST NOT create circular dependencies between profiles, where a profile inherits from itself directly.

This is an example of the parent base-profile used in the previous example, it contains a single extension "rppOther" version 1.0 and does not include any other profiles:

```
{
  "name": "urn:ietf:params:rpp:profile:base-profile",
  "description": "A base profile for provisioning objects using the RPP protocol.",
  "version": "1.0",
  "rpp_version": "1.0",
  "objects": ["domains", "hosts", "entities"],
  "extensions": [
    {
      "rppOther": {
        "version": "1.0"
      }
    }
  ],
  "profile":
}
```

The example profile definition shown in Section 12.1, Paragraph 3 uses the "base-profile" as its base and inherits its features and also overrides the version of the "rppOther" extension defined in the base profile.

12.3. Signalling

This document descibes two distinct methods for signalling the profile used in a RPP request or response, the first method uses a dedicated HTTP header, the second method uses media type parameters. The two methods MUST not be used simultaneously in a single request or response. If both methods are used in a single request or response, then the server MUST return an HTTP error response and include a Problem Detail response in the message body.

12.3.1. Header signalling

The client MUST use the `RPP-Profile` header to indicate the name of the profile that is to be used for the request. The value of this header MUST be of the type `parameter` described in [RFC8941], the first parameter MUST uniquely identify a profile, for example `urn:ietf:params:rpp:profile:example-profile`, followed by the version parameter. If the server does not support the indicated profile or version, then the server MUST return an HTTP error response and include a Problem Detail response in the message body.

The ABNF for Profile header value is as follows:

```
profile-header = "profile" "=" profile-name ";" OWS "version" "=" version
profile-name   = token
version        = 1*DIGIT "." 1*DIGIT
```

Example:

```
RPP-Profile: profile=urn:ietf:params:rpp:profile:example-profile;version=1.0
```

12.3.2. Media type parameter signalling

When using Media type parameter signalling, the client and the server MUST use media type parameters in the Accept and Content-Type headers to indicate the name and version of the profile used in the request. The media type parameters MUST be defined as follows:

- `profile`: The value of this parameter MUST uniquely identify the profile, for example `urn:ietf:params:rpp:profile:example-profile`.
- `version`: The value of this parameter MUST indicate the version of the profile used in the request.

The ABNF for media type parameter signalling is as follows:

```
profile-parameter = "profile" "=" profile-name ";" OWS "version" "=" version
profile-name      = token
version           = 1*DIGIT "." 1*DIGIT
```

Example for the media type `application/rpp+json` with profile parameters indicating the use of the "example-profile" profile version 1.0.:

```
Accept: application/rpp+json; profile="urn:ietf:params:rpp:profile:example-profile";
version="1.0"
Content-Type: application/rpp+json; profile="urn:ietf:params:rpp:profile:example-profile";
version="1.0"
```

13. Endpoints

Endpoints are described using URI Templates [RFC6570] relative to a discoverable base URL, as recommended by [RFC9205]. Some RPP endpoints do not require a request and/or response message.

The RPP endpoints are defined using the following URI Template syntax:

- {c}: An abbreviation for {collection}: this MUST be substituted with "domains", "hosts", "entities" or another collection of objects.
- {i}: An abbreviation for an object identifier, this MUST be substituted with the value of a domain name, hostname, contact-id or a message-id or any other defined object.

A RPP client MAY use the HTTP GET method for executing informational request only when no request data has to be added to the HTTP message body. Sending content using an HTTP GET request is discouraged in [RFC9110], there exists no generally defined semantics for content received in a GET request. When an RPP object requires additional information, the client MUST use the HTTP POST method and add the query command content to the HTTP message body.

13.1. Availability for Creation

The Availability for Creation endpoint is used to check whether an object can be successfully provisioned. Two distinct methods are defined for checking the availability of provisioning of an object, the first method uses the HEAD method for a quick check to find out if the object can be provisioned. The second method uses the GET method to retrieve additional information about the object's availability for provisioning, for example about pricing or additional requirements to be able to provision the requested object.

When the client uses the HTTP HEAD method, the server MUST respond with an HTTP status code 200 (OK) if the object can be provisioned or with an HTTP status code 404 (Not Found) if the object cannot be provisioned.

When the client uses the HTTP GET method, the server MUST respond with an HTTP status code 200 (OK) if the object can be provisioned. The server MUST include a message body containing more detailed availability information, for example about pricing or additional requirements to be able to provision the requested object. The message body MAY be and empty JSON object if no additional information is applicable.

If the object cannot be provisioned then the server MUST return an HTTP status code 404 (Not Found) and include a problem statement in the message body.

As an extension point the server MAY define and the client MAY use additional HTTP query parameters to further specify the check operation or the kind of response information that shall be returned. For example Registry Fee Extension [RFC8748] defines a possibility to request certain currency, only certain commands or periods. Such functionality would add query parameters, which could be used with GET request to receive additional pricing information with the response. HEAD request would not be affected in this case.

The server MUST respond with the same HTTP status code if the same URL is requested with HEAD and with GET.

```
- Request: HEAD|GET {collection}/{id}/availability
- Request message: None
- Response message: Optional availability response
```

Example request for a domain name that is not available for provisioning:

```
HEAD domains/foo.example/availability HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/2 404 Not Found
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
RPP-Cltrid: ABC-12345
RPP-Svtrid: XYZ-12345
RPP-code: 01000
Content-Length: 0
```

13.2. Resource Information

The Object Info request MUST use the HTTP GET method on a resource identifying an object instance. If the object has authorization information attached then the client MUST use an empty message body and include the RPP-Authorization HTTP header. If the authorization is linked to a database object the client MUST also include the roid in the RPP-Authorization header. The client MAY also use a message body that includes the authorization information, the client MUST then not use the RPP-Authorization header.

- Request: GET {collection}/{id}
- Request message: Optional
- Response message: Info response

Example request for an object not using authorization information.

```
GET domains/foo.example HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example request using RPP-Authorization header for an object that has attached authorization information.

```
GET domains/foo.example HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
RPP-Authorization: authinfo value=TXkgU2VjcmV0IFRva2Vu
```

Example Info response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 424
Content-Type: application/rpp+json
Content-Language: en
RPP-code: 01000

TODO: JSON message here
```

13.3. Poll for Messages

The messages endpoint is used for retrieving messages stored on the server for the client to process.

- Request: GET /messages
- Request message: None
- Response message: Poll response

The client MUST use the HTTP GET method on the messages resource collection to request the message at the head of the queue.

Example request:

```
GET messages HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 312
Content-Type: application/rpp+json
Content-Language: en
RPP-code: 01301

TODO
```

13.4. Delete Message

- Request: DELETE /messages/{id}

- Request message: None
- Response message: Poll Ack response

The client MUST use the HTTP DELETE method to acknowledge receipt of a message from the queue. The "msgID" attribute of a received RPP Poll message MUST be included in the message resource URL, using the {id} path element. The server MUST use RPP headers to return the RPP result code and the number of messages left in the queue. The server MUST NOT add content to the HTTP message body of a successful response, the server may add content to the message body of an error response.

Example request:

```
DELETE messages/12345 HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Language: en
RPP-code: 01000
RPP-Queue-Size: 0
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
Content-Length: 145

TODO
```

13.5. Create Resource

- Request: POST {collection}
- Request message: Object Create request
- Response message: Object Create response

The client MUST use the HTTP POST method to create a new object resource. If the RPP request results in a newly created object, then the server MUST return HTTP status code 200 (OK). The server MUST add the "Location" header to the response, the value of this header MUST be the URL for the newly created resource.

Example Domain Create request:

```
POST domains HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
Accept-Language: en
Content-Length: 220

TODO
```

Example Domain Create response:

```
HTTP/2 200
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Language: en
Content-Length: 642
Content-Type: application/rpp+json
Location: https://rpp.example/domains/foo.example
RPP-code: 01000

TODO
```

13.6. Delete Resource

- Request: DELETE {collection}/{id}
- Request message: Optional
- Response message: Status

The client MUST the HTTP DELETE method and a resource identifying a unique object instance. The server MUST return HTTP status code 200 (OK) if the resource was deleted successfully.

Example Domain Delete request:

```
DELETE domains/foo.example HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example Domain Delete response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 80
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-code: 01000

TODO
```

13.7. Processes Path Segment

Each provisioning object may be related to one or more running processes, such as a transfer or deletion. Each process can have its own data, which is distinct from the data of the provisioning object itself. The processes can be started, stopped, or interacted with using their own specific set of representations and operations.

All processes related to a provisioning object in RPP MUST exist under the `/{collection}/{id}/processes/{process_name}` path.

The server operator MAY support direct access to process resources using server generated identifier. Such resource MAY be accessible using following URL: `/{collection}/{id}/processes/{process_name}/{process_id}`, where process_id is the process identifier.

A process MAY also expose a resource at `/{collection}/{id}/processes/{process_name}/latest` to access and interact with the latest process instance. In case server offers any access to process information of given process name, the access to the last instance using `/{collection}/{id}/processes/{process_name}/latest` URL is MANDATORY.

The server operator MAY decide which processes such resources exist for, whether they only exist for the currently running processes or also for completed or cancelled processes. The period for which completed processes remain available for retrieval is defined by server policy.

13.7.1. Generic proces interface

A generic interface for interacting with the processes is defined as follows:

13.7.1.1. Starting:

`POST /{collection}/{id}/processes/{process_name}`

The payload of such a request contains process-specific input information. A started process MAY create a resource to access and interact with the process instance. In such case the response MUST be a 201 Created with a `Location` header pointing to the created resource together with the process state representation. The created resource can be made accessible both using the latest mnemonic under a URL `/{collection}/{id}/processes/{process_name}/latest` or using a process id under a URL `/{collection}/{id}/processes/{process_name}/{process_id}`.

When a process is created, executed and immediately completed by the server, a 201 Created response MAY still be provided together with the representation of the process result.

Server MAY decide not to expose any resource for interaction with the created process, in such case a 200 OK MUST be provided.

Example:

```
POST /rpp/v1/domains/foo.example/processes/renewals HTTP/2
... other headers removed for bravity ...

{
   "duration": "P2Y"
}
```

13.7.1.2. Cancelling:

A client MAY use the "latest" mnemonic to cancel the latest process instance, in such case the request MUST be:

`DELETE /{collection}/{id}/processes/{process_name}/latest`

If the client wants to cancel a specific process instance, the request MUST be:

`DELETE /{collection}/{id}/processes/{process_name}/{process_id}`

This request is intended to stop the running process. The server MUST return a 204 response if the process has been stopped and the resource is gone, or a 200 response if the process has been stopped but the resource remains.

13.7.1.3. Status

A client MAY use the "latest" mnemonic to request the latest process instance, in such case the request MUST be:

`GET /{collection}/{id}/processes/{process_name}/latest`

If the client wants to retrieve data of a specific process instance, the request MUST be:

`GET /{collection}/{id}/processes/{process_name}/{process_id}`

The request retrieves the representation of the task status. If no task is running, the server MAY return the status of the completed task or return a 404 response.

13.7.1.4. Other operations

Other operations on a process can be performed by adding path segments to the `/{collection}/{id}/processes/{process_name}/latest` or `/{collection}/{id}/processes/{process_name}/{process_id}` URL path.

13.7.1.5. Listing

A server MAY implement a listing facility for some or all, current or past processes.

The following URL structure and HTTP method MAY be exposed by the server and MUST be used by the client to retrieve process list filtered by process name:

`GET /{collection}/{id}/processes/{process_name}/`

The following URL structure and HTTP method MAY be exposed by the server and MUST be used by the client to retrieve full process list independent of the process name:

`GET /{collection}/{id}/processes/`

It is up to server policy to define the type of processes and state, running or completed, made available for the client. A server MAY also choose not implement this end point at all returning either the HTPP status code 404 Not Found or a 501 Not Implemented status code.

13.7.2. Relation to object representation

In certain situations a resource creation may require additional process data or implicitly start an asynchronous process with own inputs, lifecycle and state. In these cases, the representation sent to the server MAY contain a combination of object data and process-related data. For example a domain create request contains domain representation data which will be stored with domain object, and domain creation process data such as registration duration or price, which would be part as registration process data, but not directly stored with the domain object.

For the process data in the message body to be distinct and consistent with the URL path structure, it MUST be enclosed in the `processes/{process_name}` JSON path when transmitted with the object's representation.

Structure:

```
POST /.../{collection}/{id}
...
{
   ... object data ...
   "processes": {
      "{process_name}": {
         ... process data ...
      }
   }
   ...
}
```

Example: Domain Create request with 2-year registration:

```
POST /rpp/v1/domains HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
Accept-Language: en
Content-Length: 220

{
   "name": "foo.example",
   "processes": {
      "creation": {
         "periods": "P2Y"
      }
   }
   ... other domain data ...
}
```

13.8. Renew Resource

- Request: POST /{collection}/{id}/processes/renewals
- Request message: Renew request
- Response message: Renew response

Not every object resource includes support for the renew command. The response MUST include the Location header for the created renewal process resource.

Example Domain Renew request:

```
POST /rpp/v1/domains/foo.example/processes/renewals HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
RPP-Cltrid: ABC-12345
Accept-Language: en
Content-Length: 210

TODO: add renew request data here
```

Example Renew response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Language: en
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
Content-Length: 205
Location: https://rpp.example/rpp/v1/domains/foo.example/processes/renewals/
XYZ-12345
Content-Type: application/rpp+json
RPP-code: 01000

TODO add renew response data here
```

13.9. Transfer Resource

The Transfer command is mapped to a nested resource, named "transfer". The semantics of the HTTP DELETE method are determined by the role of the client executing the DELETE method. The DELETE method is defined as "reject transfer" for the current sponsoring client of the object. For the new sponsoring client the DELETE method is defined as "cancel transfer".

13.9.1. Start

- Request: POST /{collection}/{id}/processes/transfers
- Request message: Optional
- Response message: Status

In order to initiate a new object transfer process, the client MUST use the HTTP POST method on a unique resource to create a new transfer resource object. Not all RPP objects support the Transfer command.

If the transfer request is successful, then the response MUST include the Location header for the object being transferred.

Example request not using object authorization:

```
POST /rpp/v1/domains/foo.example/processes/transfers HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 0
```

Example request using object authorization:

```
POST /rpp/v1/domains/foo.example/processes/transfers HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
RPP-Cltrid: ABC-12345
RPP-Authorization: authinfo value=TXkgU2VjcmV0IFRva2Vu
Accept-Language: en
Content-Length: 0
```

Example request using 1 year renewal period, using the `unit` and `value` query parameters:

```
POST /rpp/v1/domains/foo.example/processes/transfers HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 23

{
  "duration": "P2Y"
}
```

Example Transfer response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Language: en
Content-Length: 182
Content-Type: application/rpp+json
Location: https://rpp.example/rpp/v1/domains/foo.example/processes/transfers/latest
RPP-code: 01001

{
  "trStatus": "pending",
  "reID": "ClientX",
  "acID": "ClientY",
  "reDate": "2000-06-06T22:00:00.0Z",
  "acDate": "2000-06-11T22:00:00.0Z",
  "exDate": "2002-09-08T22:00:00.0Z
}
```

13.9.2. Status

A transfer object may not exist, when no transfer has been initiated for the specified object. The client MUST use the HTTP GET method and MUST NOT add content to the HTTP message body.

- Request: GET {collection}/{id}/processes/transfers
- Request message: Optional
- Response message: Transfer Status response

Example domain name Transfer Status request without authorization information required:

```
GET /rpp/v1/domains/foo.example/processes/transfers HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

If the requested transfer object has associated authorization information that is not linked to another database object, then the HTTP GET method MUST be used and the authorization information MUST be included using the RPP-Authorization header.

Example domain name Transfer Query request using RPP-Authorization header:

```
GET /rpp/v1/domains/foo.example/processes/transfers HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
RPP-Authorization: authinfo value=TXkgU2VjcmV0IFRva2Vu
```

If the requested object has associated authorization information linked to another database object, then the HTTP GET method MUST be used and the RPP-Authorization header MUST be included.

Example domain name Transfer Query request and authorization using RPP-Authorization header:

```
GET /rpp/v1/domains/foo.example/processes/transfers HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Authorization: authinfo value=TXkgU2VjcmV0IFRva2Vu
Content-Length: 0
```

Example Transfer Query response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 230
Content-Type: application/rpp+json
Content-Language: en
RPP-code: 01000

TODO
```

13.9.3. Cancel

- Request: POST /{collection}/{id}/processes/transfers/cancelation
- Request message: Optional
- Response message: Status

The new sponsoring client MUST use the HTTP POST method to cancel a requested transfer.

Example request:

```
POST /rpp/v1/domains/foo.example/processes/transfers/cancelation HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 80
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-code: 01000

TODO
```

13.9.4. Reject

- Request: POST /{collection}/{id}/processes/transfers/rejection
- Request message: None
- Response message: Status

The currently sponsoring client of the object MUST use the HTTP POST method to reject a started transfer process.

Example request:

```
POST /rpp/v1/domains/foo.example/processes/transfers/rejection HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
```

Example Reject response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 80
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-code: 01000

TODO
```

13.9.5. Approve

- Request: POST /{collection}/{id}/processes/transfers/approval
- Request message: Optional
- Response message: Status

The currently sponsoring client MUST use the HTTP POST method to approve a transfer requested by the new sponsoring client.

Example Approve request:

```
POST /rpp/v1/domains/foo.example/processes/transfers/approval HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Accept-Language: en
RPP-Cltrid: ABC-12345
Content-Length: 0
```

Example Approve response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 80
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-code: 01000

TODO
```

13.10. Update Resource

- Request: PATCH {collection}/{id}
- Request message: Object Update message
- Response message: Status

An object Update request MUST be performed using the HTTP PATCH method. The request message body MUST contain an Update message.

TODO: when using JSON, also allow for JSON patch so client can send partial update data only?

Example request:

```
PATCH domains/foo.example HTTP/2
Host: rpp.example
Authorization: Bearer <token>
Accept: application/rpp+json
Content-Type: application/rpp+json
Accept-Language: en
Content-Length: 252

TODO
```

Example response:

```
HTTP/2 200 OK
Date: Wed, 24 Jan 2024 12:00:00 UTC
Server: Example RPP server v1.0
Content-Length: 80
RPP-Svtrid: XYZ-12345
RPP-Cltrid: ABC-12345
RPP-code: 01000

TODO
```

14. Extension Framework

TODO

15. RPP Result Codes

RPP result codes are used to indicate the result of an RPP request. They are returned in the RPP-Code header of the HTTP response. The format of the RPP result code is a 5-digit string, where the first digit MUST always be "1", the second digit indicates the class of the result, and the remaining four digits indicate the specific result within that class, his allows implementers to define more specific result codes within each class. The classes of RPP result codes are designed to match the classes of HTTP status codes, to facilitate mapping between RPP result codes and HTTP status codes. The classes of RPP result codes are defined as follows:

- 11xxx: Informational
- 12xxx: Success
- 13xxx: Reserved for future use
- 14xxx: Client error
- 15xxx: Server error

The following RPP result codes are defined and used in this document:

Table 2: RPP result codes

RPP Result Code	HTTP Status Code	Description
12000	200 OK	Command completed successfully
12001	201 Created	Command completed successfully and a new resource was created

Table 2

16. Authentication and Authorization

A key design goal of RPP's authorization model is fine-grained access control. A registrar MUST be able to operate multiple user accounts within the registry, each carrying a distinct set of permissions appropriate to the user's role (e.g., read-only reporting accounts, accounts limited to a specific set of operations, or fully privileged administrative accounts). This allows registrars to implement the principle of least privilege within their own organizations without requiring separate registry-level registrar accounts. The registry's authorization server MUST support registrar self-service management of these user accounts, enabling registrars to create, modify, and revoke user credentials and their associated permission scopes without requiring manual intervention by the registry operator.

Due to the stateless nature of RPP, the client MUST include authorization credentials in each HTTP request. RPP uses OAuth 2.0 [RFC6749] for delegated authorization via Bearer tokens. Basic authentication [RFC7617] SHOULD NOT be used. The server MUST validate the Bearer token on each request and reject any request with an invalid or expired token with an appropriate HTTP status code.

16.1. OAuth 2.0

RPP MAY use OAuth 2.0 [RFC6749] as its authorization framework. OAuth 2.0 is an authorization protocol and does not perform authentication itself; authentication of the client or end-user is handled by the authorization server before a token is issued. RPP acts as an OAuth 2.0 Resource Server and MUST validate every incoming request against a Bearer token presented in the Authorization header. The registry MAY operate its own authorization server or MAY delegate to an external authorization server. Access control decisions are derived exclusively from verified claims in the presented JWT access token.

Access tokens MUST be JWTs conforming to the JWT Profile for OAuth 2.0 Access Tokens [RFC9068]. Tokens MUST be signed using asymmetric cryptography; symmetric signing algorithms (e.g., HS256) MUST NOT be used for tokens issued by external authorization servers. Short-lived tokens are RECOMMENDED and token caching and refresh strategies MUST follow the best practices defined in [RFC8725]. Sensitive claims in the JWT payload MUST be encrypted if the token is persisted to storage.

RPP operates as a Policy Enforcement Point (PEP). The authorization server that issues a token is identified by the iss claim; the RPP server MUST validate the token's signature against the issuing authorization server's public key, fetched and cached via OAuth 2.0 Authorization Server Metadata [RFC8414].

RPP supports two registry authorization modes:

- **Internal mode**: the registry operates its own authorization server and issues tokens directly to clients.

`Client → Registry Authorization Server → JWT → RPP API`

- **External mode**: the registry trusts one or more external authorization servers and validates tokens they issue. This mode is also required for federated object transfers, where tokens are issued by a registrar's own authorization server.

`Client → External Authorization Server → JWT → RPP API`

In both modes authorization is enforced identically. The `aud` claim MUST identify the RPP server as the intended audience; the RPP server MUST reject tokens where its own identifier is absent from `aud`.

16.2. Scopes

OAuth scopes are used for enforcing access control when accessing RPP resources. The server MUST define a set of scopes that can be requested by clients when obtaining access tokens. The scopes MUST be based on the principle of least privilege, allowing clients to request only the permissions they need to perform their intended operations. The server MUST also define the mapping between scopes and the specific resources and operations that they grant access to.

When a client requests an access token from the authorization server, it MUST include the desired scopes in the scope parameter of the token request. The authorization server MUST validate the requested scopes against the client's registered permissions and issue an access token with the appropriate scopes if the request is valid. The server MUST enforce access control based on the scopes included in the access token, allowing or denying access to resources based on the client's authorized scopes. The server MUST also implement appropriate error handling for cases where a client attempts to access a resource without the necessary scopes, returning an appropriate HTTP status code and error message.

RPP scopes are based on the objects, processes and operations defined in RESTful Provisioning Protocol (RPP) data objects in [I-D.kowalik-rpp-data-objects]. Each scope corresponds to a specific set of permissions for accessing and manipulating RPP resources.

16.2.1. Scope Derivation Rules

RPP scopes are derived systematically from the data object types and operation categories defined in [I-D.kowalik-rpp-data-objects]. The derivation rules are as follows:

- The scope identifier MUST use the format `<object>:<access-level>`, where `<object>` is the lowercase stable identifier of the data object and `<access-level>` is one of the access levels defined below.
- The `create` access level grants permission to perform the Create operation on the corresponding data object.
- The `read` access level grants permission to perform the Read operation on the corresponding data object.

- The update access level grants permission to perform Update, Renew, and Restore operations on the corresponding data object.
- The delete access level grants permission to perform the Delete operation on the corresponding data object.
- The transfer access level grants permission to perform all Transfer operations on the corresponding data object.
- For interactive OAuth 2.0 federated transfers, authorization MUST convey the specific object being transferred so the registrant can give informed, object-scoped consent. Two mechanisms are defined in order of preference:
  - **Primary - Rich Authorization Requests (RAR) [RFC9396]**: The gaining registrar MUST include an authorization_details parameter in the authorization request containing an object of type rpp_transfer with object_type and object_identifier fields (see Table 3). This is the RECOMMENDED approach when the losing registrar's authorization server supports RAR.
  - **Fallback - Transfer Scope**: When the losing registrar's authorization server does not support RAR, a static transfer scope of the form transfer:<object> MUST be used (e.g., transfer:domain). The specific object identifier MUST be carried in the rpp_object_id claim in the issued JWT (see Section 16.3.2). This scope MUST be presented to the registrant for approval, and the authorization server MUST include the rpp_object_id claim in the token.

- Extensions and additional data objects registered in the IANA RPP Data Object Registry [I-D.kowalik-rpp-data-objects] SHOULD define their own scopes following the same derivation rules, using the registered stable identifier of the extension object as the <object> component.

16.2.2. Scope Registry

Table Table 3 defines the RPP scopes derived from the data objects specified in [I-D.kowalik-rpp-data-objects].

Scope	Data Object	Operations Granted
domain:create	Domain Name	Create
domain:read	Domain Name	Read
domain:update	Domain Name	Update, Renew, Restore
domain:delete	Domain Name	Delete
domain:transfer	Domain Name	Transfer Create, Transfer Approve, Transfer Reject, Transfer Cancel, Transfer Query (Fallback flow only)
contact:create	Contact	Create

Scope	Data Object	Operations Granted
contact:read	Contact	Read
contact:update	Contact	Update, Restore
contact:delete	Contact	Delete
contact:transfer	Contact	Transfer Create, Transfer Approve, Transfer Reject, Transfer Cancel, Transfer Query (Fallback flow only)
host:create	Host	Create
host:read	Host	Read
host:update	Host	Update, Restore
host:delete	Host	Delete

Table 3: RPP OAuth 2.0 Scopes

16.2.3. Rich Authorization Requests (RAR)

OAuth 2.0 Rich Authorization Requests [RFC9396] extends the standard OAuth 2.0 authorization request with an `authorization_details` parameter that carries a structured JSON object describing precisely what the client is requesting authorization for. Unlike scopes, which are coarse-grained string tokens, `authorization_details` allows the request to include typed, fine-grained authorization data, such as the specific object being transferred. The authorization server can present to the user in a meaningful consent screen.

In RPP, RAR is used for interactive federated object transfers to convey the specific domain name or contact handle being transferred. The gaining registrar includes an `authorization_details` object of type `rpp_transfer` in the authorization request to the losing registrar's authorization server. The registrant then sees exactly which object they are consenting to transfer. The authorization server MUST echo the `authorization_details` object back as a claim in the issued JWT, giving the registry verifiable, tamper-proof evidence of what was authorized and for which object.

The `type` field MUST be set to `rpp_transfer`. Table Table 4 lists the RAR fields defined for RPP.

Field	Type	Requirement	Description
type	String	REQUIRED	MUST be rpp_transfer.
object_type	String	REQUIRED	The RPP object type being transferred. MUST be one of domain or contact.

Field	Type	Requirement	Description
object_identifier	String	REQUIRED	The fully qualified name or handle of the specific object to be transferred (e.g., foo.example or CID-12345).

Table 4: RPP Transfer Authorization, RAR authorization_details object (Primary Method, [RFC9396])

Example RAR authorization_details value for a domain transfer:

```
[{
  "type": "rpp_transfer",
  "object_type": "domain",
  "object_identifier": "foo.example"
}]
```

The losing registrar's authorization server MUST echo the authorization_details back as a claim in the issued JWT. The registry MUST validate the authorization_details claim in the token and MUST verify that object_type and object_identifier match the object being transferred.

When the losing registrar's authorization server does not support RAR ([RFC9396]), the specific object being transferred MUST be conveyed via the rpp_object_id claim rather than encoded in the scope string:

- To authorize a domain transfer, the specific domain name MUST be present in the rpp_object_id claim (e.g., foo.example).
- To authorize a contact transfer, the specific contact identifier MUST be present in the rpp_object_id claim (e.g., REG-12345).

16.3. Claims

RPP access tokens MUST conform to the JWT Profile for OAuth 2.0 Access Tokens defined in [RFC9068]. This profile establishes a standardized set of claims that RPP uses to make authorization decisions consistently across implementations and identity providers.

16.3.1. Required Claims

Table Table 5 lists the claims defined in [RFC9068], these MUST be present in every RPP access token:

Claim	Type	Description
iss	String (URI)	Identifies the issuing authorization server. The RPP server MUST validate this against its set of trusted issuers.

Claim	Type	Description
sub	String	The subject of the token. For machine-to-machine flows this MUST be the client identifier. For interactive flows this MUST be the end-user identifier of the authenticated end-user at the issuing authorization server. The end-user MAY be a registrar employee operating the registrar's management system, or a registrar customer (registrant) directly interacting with the registrar's client application, for example when initiating an object transfer.
aud	String or Array	Identifies the intended audience. MUST include the RPP server's resource identifier. The RPP server MUST reject tokens where its own identifier is not present in this claim.
exp	Numeric date	Expiry time. The RPP server MUST reject tokens that have expired.
iat	Numeric date	Time at which the token was issued.
jti	String	Unique identifier for the token, used to prevent token replay attacks.
client_id	String	The OAuth 2.0 client identifier of the gaining registrar or RPP client application.
scope	String	Space-separated list of granted scopes (see Section 16.2). The RPP server MUST enforce access control based on the scopes present in this claim.

Table 5: OAuth 2.0 Access Token Claims for RPP ([RFC9068])

16.3.2. RPP-Specific Claims

In addition to the standard [RFC9068] claims, table Table 6 lists the RPP-specific claims that are defined to enable fine-grained authorization decisions. Required claims MUST be present in every RPP access token. Optional claims SHOULD be included when applicable to the deployment or request context.

Claim	Requirement	Type	Description
rpp_registrar_id	REQUIRED	String	The identifier of the registrar on whose behalf the request is made. The RPP server MUST validate that this identifier matches a known and authorized registrar. This claim MUST be present in all access tokens used for RPP requests.

Claim	Requirement	Type	Description
rpp_reseller_id	OPTIONAL	String	The identifier of the reseller acting through the registrar's client application. This claim SHOULD be included when the request originates from a reseller operating under the registrar's account. The RPP server MAY use this claim for access control, auditing, and attribution purposes. The value is interpreted within the namespace of the registrar identified by rpp_registrar_id.
rpp_transfer_authinfo	OPTIONAL	String	The transfer authorization information used in Fallback flow object transfer. When present, the value MUST be equivalent to the value that would otherwise be sent in the RPP-Authorization header, using the same <method> <authorization information> format. The RPP server MUST treat this claim as equivalent to the RPP-Authorization header; if both are present in the same request, the claim value MUST take precedence. This claim MUST only be present in access tokens used for transfer operations.
rpp_object_id	OPTIONAL	String	The identifier of the specific object being transferred. MUST be present in access tokens for interactive federated transfers when the fallback dynamic scope method is used (transfer:domain or transfer:contact) and the authorization_details claim ([RFC9396]) is not present. The value MUST be the fully qualified name or handle of the object (e.g., foo.example or CID-12345). The RPP server MUST validate that this value matches the object being transferred. This claim MUST NOT be present when authorization_details is present.

Claim	Requirement	Type	Description
authorization_details	OPTIONAL	Array of Objects	Rich authorization details as defined by [RFC9396]. MUST be present in access tokens for interactive federated transfers when the RAR method is used. Each object in the array MUST have a type field; for RPP transfer tokens the type MUST be rpp_transfer and the object MUST include object_type and object_identifier fields (see Table 4). The RPP server MUST validate that object_type and object_identifier match the object being transferred. When this claim is present, the rpp_object_id claim MUST NOT be present.

Table 6: RPP Specific Access Token Claims

The combination of the sub claim and the rpp_registrar_id claim provides a complete, two-dimensional identity for every RPP request: sub identifies the individual principal (registrar employee, automated process, or registrar customer) that initiated the request, while rpp_registrar_id identifies the registrar organization as a whole within the registry's domain.

The identity of the sub depends on both the flow type and the identity domain of the principal:

- In **machine-to-machine** flows, sub is the registrar's OAuth 2.0 client identifier, representing an automated system acting on behalf of the registrar. The token is issued by the **registry's authorization server**, which maintains the registrar's client credentials.
- In **interactive flows initiated by registrar staff**, sub is the identifier of the registrar employee. Registrar employees are managed as users in the **registry's authorization server**. The token is therefore also issued by the registry's authorization server, and the sub value is the employee's account identifier within that server. The iss claim will identify the registry's authorization server.
- In **interactive flows initiated by a registrant customer**, the situation is different. Registrant customers are not managed in the registry's authorization server, they are maintained in the **registrar's own authorization server** (the registrar acts as the authorization server for its customers). The token is therefore issued by the registrar's authorization server, and the iss claim will identify the registrar's authorization server as the issuer. The registry MUST have a pre-established trust relationship with the registrar's authorization server to accept and validate such tokens. In this case, the sub value MUST be the registrant's identifier as it exists in the registry database. The registrar MUST use this registry-assigned id, not any registrar-internal customer identifier, as the sub value. This ensures the registry can unambiguously correlate the token's subject to an existing provisioned contact object. This enables verification of ownership and consent for operations. The registry MUST reject tokens where the sub

value does not match a known contact handle associated with the object being acted upon.

Together, these claims allow the RPP server to:

- **Enforce access control** at both the organizational level (is this registrar authorized to manage this object?) and the individual level (has this principal been granted the necessary permissions?).
- **Attribute actions to individuals** for audit trail purposes, enabling the registry to record not just which registrar performed an operation, but which employee or customer initiated it.
- **Support delegation models**, where a registrar may grant different employees different scopes (e.g., a junior employee may hold only `domain:read` scope while a senior employee holds `domain:create` and `domain:update`).
- **Verify registrant consent** in interactive transfer flows, where a `sub` containing the registrant's handle provides the registry with verifiable evidence that the object owner explicitly authorized the transfer.
- **Facilitate incident response**, allowing the registry to correlate suspicious activity back to a specific principal rather than only to a registrar organization.

The RPP server SHOULD log both the `sub` and `rpp_registrar_id` claims for every request in its audit log. The `sub` value is only meaningful within the namespace of the issuing authorization server identified by the `iss` claim. The RPP server MUST NOT compare `sub` values across different issuers, except when the `sub` is a registry contact handle, in which case the registry MAY validate it against its own database regardless of issuer.

Extensions and profiles MAY define additional claims. All additional claims MUST use a URI or a collision-resistant name as the claim name to prevent conflicts with registered claims.

16.3.3. Claim Validation

The RPP server MUST validate all required claims in accordance with [RFC9068] and [RFC8725]. Specifically:

- The `iss` claim MUST identify a trusted authorization server whose public keys are known to the RPP server, either through static configuration or dynamic discovery (e.g., OAuth 2.0 Authorization Server Metadata [RFC8414]).
- The `aud` claim MUST be validated to confirm the token is intended for this RPP server.
- The `exp` claim MUST be checked and expired tokens MUST be rejected with HTTP 401 Unauthorized.
- The JWT signature MUST be verified using asymmetric cryptography (e.g., RS256 or ES256). Symmetric algorithms (e.g., HS256) MUST NOT be used for tokens issued by external authorization servers.
- If the `scope` claim is absent or does not contain the scope required for the requested operation, the RPP server MUST return HTTP 403 Forbidden.

16.4. Flows

RPP defines two authorization flows: machine-to-machine flows and interactive flows. Machine-to-machine flows are designed for use in automated systems where no user interaction is required. Interactive flows are designed for use in scenarios where end-user interaction is required, such as when a registrant initiates a transfer request using a registrar's web interface.

16.4.1. Machine to Machine

The machine-to-machine (M2M) flow is used for automated, unattended RPP requests such as domain provisioning, renewal, or host management sent by a registrar's backend systems to the registry. No end-user interaction is involved. The OAuth 2.0 Client Credentials grant Section 4.4 MUST be used for this flow.

In this flow the registrar's system authenticates directly with the registry's authorization server using a pre-registered client_id and client_secret (or a private key JWT for higher assurance). The authorization server issues a short-lived access token scoped to the requested RPP operations. The registrar's system then includes this token in the Authorization header of each RPP request sent to the registry.

The sub claim in the resulting token MUST be set to the client_id of the registrar's system. The rpp_registrar_id claim MUST also be present, identifying the registrar organization within the registry's namespace.

```
Registrar              Registry          Registry
Backend                 Auth Server        RPP Server
  |                    |                |
  | 1. Token request        |                |
  |  (client_id +           |                |
  |   client_secret         |                |
  |   + scope)              |                |
  +------------------------->|                |
  |                    |                |
  | 2. Access token (JWT)     |                |
  |<-------------------------|                |
  |                    |                |
  | 3. RPP request          |                |
  |  (Authorization:        |                |
  |   Bearer <token>)        |                |
  +----------------------------------------------->|
  |                    |                |
  |                    |  4. Validate JWT   |
  |                    |  (verify signature |
  |                    |   using cached     |
  |                    |   public key,      |
  |                    |   check claims     |
  |                    |   and scopes)      |
  |                    |                |
  | 5. RPP response         |                |
  |<-----------------------------------------------|
  |                    |                |
```

Figure 1: OAuth 2.0 Client Credentials Flow for Registrar-to-Registry Requests

The steps in the diagram are as follows:

1. The registrar's backend system sends a token request to the registry's authorization server, providing its client_id, client_secret (or a signed client assertion), and the requested RPP scopes (e.g., domain:create).
2. The authorization server validates the client credentials and issues a signed, short-lived JWT access token containing the granted scopes, sub (set to client_id), and rpp_registrar_id.
3. The registrar's system sends the RPP request to the registry's RPP server, including the access token in the HTTP Authorization header as a Bearer token.
4. The RPP server validates the JWT entirely locally, without contacting the authorization server. It verifies the token's signature using the authorization server's public key (previously fetched and cached via OAuth 2.0 Authorization Server Metadata [RFC8414] and the referenced JWKS endpoint), checks the standard claims (iss, aud, exp), and confirms that the scope claim includes the scope required for the requested operation.
5. If validation succeeds, the RPP server processes the request and returns the RPP response.

It is RECOMMENDED that access tokens be short-lived (e.g., minutes to hours) and that the registrar's system obtain a new token before the current token expires rather than waiting for a 401 response. Token caching and refresh strategies SHOULD follow the best practices in [RFC8725].

16.4.2. Interactive

The interactive flow is used for RPP requests that require end-user interaction. The OAuth 2.0 Authorization Code grant Section 4.1 MUST be used for this flow. Two distinct types of end-users may initiate interactive RPP requests, and they are managed in different authorization servers:

- **Registrar employees** are managed as users in the **registry's authorization server**. A registrar employee uses the registrar's client application to perform operations on behalf of the registrar, such as updating domain records or managing contacts. The registry's authorization server authenticates the employee and issues an access token. The `sub` claim will contain the employee's account identifier in the registry's authorization server, and the `iss` claim will identify the registry's authorization server.
- **Registrant customers** are managed in the **registrar's own authorization server**. A registrant customer interacts directly with the registrar's client application, for example to authorize a domain transfer. The registrar's authorization server authenticates the customer and issues an access token. The `sub` claim MUST be set to the registrant's contact handle as registered in the registry database. The `iss` claim will identify the registrar's authorization server. The registry MUST have a pre-established trust relationship with the registrar's authorization server to accept tokens it issues.

16.4.2.1. Registrar Employee Flow

In this flow, the employee authenticates with the registry's authorization server. The registry acts as both the authorization server and the RPP resource server.

```
Registrar         Registry           Registry
Employee +        Auth Server        RPP Server
Client App          |                  |
  |               |                |
  | 1. Login /      |                |
  |  auth request     |                |
  +------------------>|                |
  |               |                |
  | 2. Access token   |                |
  |  (sub=employee_id |                |
  |   iss=registry)   |                |
  |<------------------|                |
  |               |                |
  | 3. RPP request    |                |
  |  (Bearer token)   |                |
  +---------------------------------------->|
  |               |                |
  |               |  4. Validate JWT   |
  |               |  (local, cached    |
  |               |   registry public  |
  |               |   key)             |
  |               |                |
  | 5. RPP response   |                |
  |<----------------------------------------|
  |               |                |
```

Figure 2: Interactive Flow - Registrar Employee (Registry Auth Server)

16.4.2.2. Registrant Customer Flow

In this flow, the registrant authenticates with the registrar's own authorization server. The registry must trust the registrar's authorization server and validate the sub against its contact database.

```
Registrant        Registrar          Registry
Customer +          Auth Server          RPP Server
Client App            |                  |
  |               |                  |
  | 1. Login /      |                  |
  |  auth request     |                  |
  +------------------>|                  |
  |               |                  |
  | 2. Access token   |                  |
  |  (sub=contact     |                  |
  |   handle,         |                  |
  |   iss=registrar)  |                  |
  |<------------------|                  |
  |               |                  |
  | 3. RPP request    |                  |
  |  (Bearer token)   |                  |
  +---------------------------------------->|
  |               |                  |
  |               |  4. Validate JWT   |
  |               |  (local, cached    |
  |               |   registrar public |
  |               |   key via          |
  |               |   federation)      |
  |               |                  |
  |               |  5. Verify sub     |
  |               |  matches contact   |
  |               |  handle in         |
  |               |  registry DB       |
  |               |                  |
  | 6. RPP response   |                  |
  |<----------------------------------------|
  |               |                  |
```

Figure 3: Interactive Flow - Registrant Customer (Registrar Auth Server)

The steps for the registrant customer flow are as follows:

1. The registrant authenticates with the registrar's authorization server through the registrar's client application, using the Authorization Code grant.
2. The registrar's authorization server issues a signed JWT access token. The `sub` claim MUST be set to the registrant's contact handle as it exists in the registry database. The `iss` claim identifies the registrar's authorization server. The `rpp_registrar_id` claim MUST be present, identifying the sponsoring registrar.
3. The registrar's client application sends the RPP request to the registry's RPP server, including the access token as a Bearer token.
4. The RPP server validates the JWT locally using the registrar's authorization server public key, previously obtained via federation metadata. No live call to the registrar's authorization server is needed.
5. The RPP server verifies that the `sub` value matches a known contact handle in the registry database and that the contact is associated with the object being acted upon.
6. If all validation succeeds, the RPP server processes the request and returns the response.

16.4.3. Enforcing Interactive Authentication for High-Risk Operations

The machine-to-machine (M2M) Client Credentials flow is appropriate for routine automated provisioning, but certain high-risk operations, such as object transfer or modification of authorisation information, MUST require a token demonstrating that a human principal explicitly authenticated and authorized the action. Without enforcement, a registrar could use M2M tokens for all operations, eliminating individual accountability.

The following mechanisms MAY be used by the registry to enforce interactive authentication for designated operations:

sub MUST identify a human principal: The registry MAY require that for high-risk operations, the `sub` claim MUST contain the identifier of an authenticated human principal, either a registrar employee or a registrant customer, and MUST NOT equal the `client_id`. In M2M tokens issued via the Client Credentials grant, `sub` is always set to `client_id`, representing an automated system rather than a human. By mandating `sub` ≠ `client_id` for designated operations, the registry ensures those operations can only be performed with a token issued on behalf of a real, identified individual. The RPP server MUST reject requests for these operations when `sub` equals `client_id`.

Scope restriction by grant type: The registry's authorization server SHOULD be configured to refuse issuing certain high-risk scopes (e.g., `domain:transfer`) to the Client Credentials grant type. Only the Authorization Code grant (interactive) MAY be permitted to obtain these scopes. This prevents a registrar from obtaining the necessary scope for a high-risk operation through unattended M2M authentication.

The set of operations for which interactive authentication is required is a matter of registry policy and MUST be discoverable using the RPP discovery mechanism.

16.5. Secure Object Transfer

NOTE: this is very early draft of how a transfer could work in RPP, it is added here to provide a complete picture of the transfer flow and to be able to discuss the details of the transfer flow and the security mechanisms that can be used to secure the transfer flow. This section is expected to be significantly revised and updated as the transfer flow and the security mechanisms are further developed and refined.

The server MAY support a secure object transfer mechanism based on OAuth 2.0 federation for secure object transfer between registrars, the gaining registrar can obtain an access token from the losing registrar's authorization server to authorize the transfer request. This provides a secure mechanism for transferring objects without exposing sensitive information in the transfer request or response messages. This also prevents the use of opaque transfer tokens and their inherent security risks, such as token leakage or replay attacks. If any of the parties in the transfer flow does not support the secure transfer mechanism based on OAuth 2.0 federation, then the transfer MUST fall back to the Fallback flow transfer mechanism using opaque transfer tokens, using the `RPP-Authorization` header to include the transfer token in the transfer request messages.

The two transfer mechanisms cover complementary scenarios:

- **Interactive flow (OAuth 2.0 federated):** The current registrant has an account at the losing registrar and actively initiates or approves the transfer themselves — for example, a registrant migrating their own domain to a new registrar. The registrant authenticates at the losing registrar's authorization server and grants explicit, object-scoped consent.
- **Fallback flow (authinfo):** The losing registrar has not registered an authorization server URI with the registry, or the domain has been sold and the new owner has no account at the losing registrar. In this case interactive consent at the losing registrar is not possible. The seller (the previous owner, who does have an account at the losing registrar) obtains a transfer authorization code (authinfo) out-of-band and passes it to the buyer. The buyer provides the authinfo to the gaining registrar, which includes it in the transfer request via the RPP-Authorization header or the rpp_transfer_authinfo JWT claim. The registry validates the authinfo and executes the transfer without requiring the new owner to have any account at the losing registrar.

Flow selection is not at the discretion of the gaining registrar. The gaining registrar MUST use whichever flow the losing registrar's capabilities dictate, as determined by registry discovery. Specifically:

- If the registry's discovery endpoint returns an authorization server URI for the losing registrar, the gaining registrar MUST use the interactive OAuth 2.0 federated flow. Fallback flow MUST NOT be used as a substitute, even if it would be simpler to implement. The only exception is if the new owner is a different party than the previous owner (e.g., the domain was sold), in which case the gaining registrar MUST use Fallback flow because the new owner cannot authenticate to the losing registrar's authorization server to give consent.
- If the registry's discovery endpoint returns no authorization server URI for the losing registrar, the gaining registrar MUST use Fallback flow.

The registry MUST enforce this: if a Fallback flow transfer request (i.e., a request carrying an RPP-Authorization header or rpp_transfer_authinfo claim) is received for an object whose losing registrar has a registered authorization server URI, the registry MUST reject the request with an appropriate error response. This prevents downgrade attacks and ensures that registrars cannot bypass the stronger OAuth 2.0 flow for operational convenience once they have declared support for it.

For the secure transfer mechanism based on OAuth 2.0 federation, the RPP server requires an established trust relationship with the authorization server of the losing registrar. The server MUST support the necessary mechanisms for validating tokens issued by the losing registrar's authorization server. this requires the server to support the JWT profile for OAuth 2.0 Access Tokens [RFC9068] and to support the necessary mechanisms for validating tokens issued by external authorization servers. The client MUST include a valid access token in the Authorization header of the transfer request, and the server MUST validate the token and the associated permissions before allowing the transfer to proceed. The server MUST also implement appropriate error handling for cases where the token is invalid, expired, or does not have the necessary permissions for the requested transfer operation. Registrars implementing secure transfer mechanisms based on OAuth 2.0 federation MUST support the federated identity provider function.

16.5.1. Federation Trust Model

The RPP federation model uses the registry as the central trust anchor, operating as a hub-and-spoke topology. Registrars establish a trust relationship with the registry during accreditation; they do not need to establish direct trust relationships with each other. This allows any two registrars to participate in a federated transfer without any prior bilateral arrangement.

Registry as trust anchor. As part of registrar onboarding, each registrar that operates its own authorization server (i.e., maintains registrant accounts) MUST register its authorization server metadata with the registry. This includes at minimum:

- The authorization server's authorization endpoint URI, used by the gaining registrar to construct the redirect.
- The authorization server's JWKS endpoint URI or the public key material itself, used by the registry to validate tokens issued by that authorization server.

The registry stores this metadata as part of the registrar's profile and makes it available via the discovery mechanism.

Token validation without bilateral trust. When the registry receives a transfer request carrying a JWT issued by the losing registrar's authorization server, it validates the token locally using the losing registrar's public key that was registered at onboarding. No runtime call to the losing registrar or its authorization server is required. The registry already trusts that public key because it was registered through the accreditation process.

Gaining registrar. The gaining registrar only needs to trust the registry. It obtains the losing registrar's authorization server URI from the registry's discovery endpoint, redirects the registrant's browser there, and exchanges the resulting authorization code for a token. It does not need to pre-configure or authenticate the losing registrar. The browser redirect and back-channel token exchange are both secured by standard HTTPS and do not require a federated identity relationship between the two registrars.

The following diagram shows how authorization server URIs and public keys flow through the registry, and how the gaining registrar uses them at transfer time to redirect the registrant's browser:

```
Gaining            Registry           Losing
Registrar         (Trust Anchor)       Registrar
  |                |                |
  |                |                |
  : --- Onboarding --- : --- Onboarding --- :
  |                |                |
  | Register own AS    |                |
  | URI + JWKS pubkey  |                |
  +------------------->|                |
  |                |                |
  |                | Register own AS    |
  |                | URI + JWKS pubkey  |
  |                |<-------------------|
  |                |                |
  : --- Transfer time -------------------- :
  |                |                |
  | Discover Losing    |                |
  | Registrar AS URI   |                |
  +------------------->|                |
  |                |                |
  | AS URI returned    |                |
  |<-------------------|                |
  |                |                |
  | Redirect registrant's browser          |
  | to Losing Registrar AS                 |
  +---------------------------------------->|
  |                |                |
  | Auth code returned via browser redirect |
  |<----------------------------------------|
  |                |                |
  | Exchange auth code |                |
  | for access token   |                |
  | (back-channel)     |                |
  +---------------------------------------->|
  |                |                |
  | JWT (signed by     |                |
  | Losing Reg AS)     |                |
  |<----------------------------------------|
  |                |                |
  | Transfer request   |                |
  | + Bearer JWT       |                |
  +------------------->|                |
  |                |                |
  |                | Validate JWT using |
  |                | cached Losing Reg  |
  |                | JWKS pubkey        |
  |                | (no runtime call)  |
  |                |------------------> |
  |                |                |
  | Transfer result    |                |
  |<-------------------|                |
  |                |                |
```

Figure 4: RPP Federation Trust Model — Onboarding, Discovery and Browser Redirect

Security properties. This model provides the following guarantees:

- A rogue registrar cannot forge a transfer token that the registry will accept, because only the legitimate losing registrar's public key (registered at onboarding) can produce a valid signature.
- The gaining registrar cannot forge registrant consent, because the token is issued by the losing registrar's authorization server, not the gaining registrar.
- The registry controls the set of trusted authorization servers by controlling which registrar IdP metadata it accepts at onboarding.
- Registrars need no knowledge of each other beyond what the registry exposes via the discovery endpoint.

16.5.2. Interactive Flow

The interactive flow uses the OAuth 2.0 Authorization Code grant Section 4.1 to obtain explicit, object-specific registrant consent directly from the losing registrar's authorization server. Before redirecting the registrant, the gaining registrar MUST first query the registry's registrar discovery endpoint to resolve the losing registrar's authorization server's authorization URI. If no authorization server URI is available for the losing registrar, the gaining registrar MUST fall back to the Fallback flow described in Section 16.5.3.

The authorization request MUST convey the specific object being transferred so that the losing registrar's authorization server can present the registrant with an accurate consent screen. The **primary method** is Rich Authorization Requests (RAR) [RFC9396]: the gaining registrar MUST include an `authorization_details` parameter of type `rpp_transfer` (see Table 4). When the losing registrar's authorization server does not support RAR, the **fallback method** is to request a dynamic transfer scope of the form `<object>:transfer` (e.g., `domain:transfer`) and the object identifier MUST be added to the `rpp_object_id` claim. In either case, the resulting token MUST be bound to the single identified object and MUST NOT be accepted by the registry for any other object.

The following diagram illustrates the interactive OAuth 2.0 federated secure object transfer flow:

```
  Client         Gaining        Registry       Losing
(Registrant)    Registrar                     Registrar
    |            |            |            |
    | 1. Initiate  |            |            |
    |  transfer of  |            |            |
    |  foo.example  |            |            |
    +-------------->|            |            |
    |            |            |            |
    |            | 2. Lookup    |            |
    |            |  losing reg  |            |
    |            |  AS URI     |            |
    |            +-------------->|            |
    |            |            |            |
    |            | 3. Return    |            |
    |            |  losing reg  |            |
    |            |  AS URI     |            |
    |            |<--------------|            |
    |            |            |            |
    | 4. Redirect  |            |            |
    |  to Losing Reg|            |            |
    |  AS (RAR:    |            |            |
    |  authz_details|            |            |
    |  type=rpp_    |            |            |
    |  transfer,    |            |            |
    |  id=example   |            |            |
    |  .com)       |            |            |
    |<--------------|            |            |
    |            |            |            |
    | 5. Auth &    |            |            |
    |  approve     |            |            |
    |  transfer at  |            |            |
    |  Losing Reg   |            |            |
    |  AS          |            |            |
    +----------------------------------------------->|
    |            |            |            |
    | 6. Auth code  |            |            |
    |  + redirect to|            |            |
    |  Gaining Reg  |            |            |
    |  callback URI |            |            |
    |<-----------------------------------------------|
    |            |            |            |
    | 7. Follow    |            |            |
    |  redirect    |            |            |
    |  (auth code   |            |            |
    |  delivered)   |            |            |
    +-------------->|            |            |
    |            |            |            |
    |            | 8. Exchange   |            |
    |            |  auth code for|            |
    |            |  access token |            |
    |            +------------------------------>|
    |            |            |            |
    |            | 9. JWT token  |            |
    |            |  (authz_     |            |
    |            |  details=    |            |
    |            |  {type:rpp_   |            |
    |            |  transfer,    |            |
```

```
     |           |  id:example   |            |
     |           |  .com})      |            |
     |           |<-----------------------------|
     |           |           |            |
     |           | 10. Transfer  |            |
     |           |  request +    |            |
     |           |  Bearer token |            |
     |           +-------------->|            |
     |           |           |            |
     |           |           | 11. Local JWT |
     |           |           |  validation   |
     |           |           |  (losing reg  |
     |           |           |  AS pubkey,  |
     |           |           |  verify scope)|
     |           |           |            |
     |           |           | 12. Execute   |
     |           |           |  transfer &   |
     |           |           |  notify Losing|
     |           |           |  Registrar    |
     |           |           +-------------->|
     |           |           |            |
     |           | 13. Transfer  |            |
     |           |   successful  |            |
     |           |<--------------|            |
     |           |           |            |
     |  14. Transfer |            |            |
     |   confirmed   |            |            |
     |<--------------|            |            |
     |           |           |            |
```

Figure 5: OAuth 2.0 Federated Secure Object Transfer - Interactive Flow

The steps in the diagram are as follows:

1. The client initiates a transfer request to the gaining registrar, specifying the object to transfer (e.g., foo.example).
2. The gaining registrar queries the registry's registrar discovery endpoint to look up the losing registrar's authorization server authorization URI. The losing registrar is identified from the current sponsoring registrar data on the domain.
3. The registry returns the losing registrar's authorization server authorization URI. If no URI is registered, the gaining registrar MUST fall back to the Fallback flow.
4. The gaining registrar redirects the client's browser to the losing registrar's authorization server. The authorization request MUST include an authorization_details parameter of type rpp_transfer with object_type: "domain" and object_identifier: "foo.example" (RAR, [RFC9396]). If the authorization server does not support RAR, the fallback dynamic scope transfer:domain:foo.example MUST be used instead. The gaining registrar's callback URI is included as the OAuth 2.0 redirect_uri.
5. The client authenticates at the losing registrar's authorization server and explicitly approves the transfer scope, providing direct, verifiable consent to release the specific object.
6. The losing registrar's authorization server issues an authorization code and redirects the client's browser back to the gaining registrar's registered callback URI.

7. The client's browser follows the redirect, delivering the authorization code to the gaining registrar's callback endpoint.
8. The gaining registrar exchanges the authorization code for an access token at the losing registrar's authorization server token endpoint (back-channel, Authorization Code grant Section 4.1).
9. The losing registrar's authorization server validates the code and issues a signed JWT access token ([RFC9068]). When RAR was used, the token MUST contain an `authorization_details` claim echoing the `rpp_transfer` object. When the dynamic scope fallback was used, the token MUST contain the `transfer:domain` (or `transfer:contact`) scope in the `scope` claim and MUST include the `rpp_object_id` claim carrying the specific object identifier.
10. The gaining registrar submits the transfer request to the registry, including the JWT as a Bearer token in the `Authorization` header.
11. The registry validates the JWT locally using the losing registrar's authorization server public key (obtained via OAuth 2.0 Authorization Server Metadata [RFC8414]). No live call to the losing registrar is required. The registry MUST verify the transfer authorization using the appropriate method: if the token contains an `authorization_details` claim (RAR), the registry MUST verify that the `type` is `rpp_transfer` and that `object_identifier` matches the object being transferred; if the token uses the dynamic scope fallback, the registry MUST verify that the `scope` claim contains `transfer:domain` or `transfer:contact` and that the `rpp_object_id` claim value matches the object being transferred.
12. The registry executes the transfer and notifies the losing registrar.
13. The registry returns a successful transfer response to the gaining registrar.
14. The gaining registrar confirms the completed transfer to the client.

16.5.3. Fallback Flow

The fallback Flow uses opaque transfer authorization tokens to authorize object transfers between registrars. The gaining registrar obtains the transfer authorization token out-of-band from the losing registrar (e.g., via the registrant or a prior registrar-to-registrar agreement) and includes it in the transfer request. No OAuth 2.0 federation or end-user interaction is required. The registry validates the opaque token against the losing registrar's records to authorize the transfer.

The transfer authorization token MUST be conveyed using the `rpp_transfer_authinfo` claim embedded in a JWT access token.

The following diagram illustrates the machine-to-machine Fallback flow for a secure object transfer:

```
  Client         Gaining        Registry       Losing
(Registrant)    Registrar                     Registrar
    |           |           |           |
    |   1. Initiate |           |           |
    |    transfer   |           |           |
    |   (authz token|           |           |
    |    provided   |           |           |
    |    out-of-band|           |           |
    |    by losing  |           |           |
    |    registrar) |           |           |
    +-------------->|           |           |
    |           |           |           |
    |           | 2. Transfer   |           |
    |           |  request      |           |
    |           |  (RPP-Authori-|           |
    |           |   zation:     |           |
    |           |   <token>)    |           |
    |           +-------------->|           |
    |           |           |           |
    |           |           | 3. Validate   |
    |           |           |  authz token  |
    |           |           |           |
    |           |           | 4. Execute    |
    |           |           |  transfer &   |
    |           |           |  notify Losing|
    |           |           |  Registrar    |
    |           |           +-------------->|
    |           |           |           |
    |           | 5. Transfer   |           |
    |           |   successful  |           |
    |           |<--------------|           |
    |           |           |           |
    |  6. Transfer  |           |           |
    |   confirmed   |           |           |
    |<--------------|           |           |
    |           |           |           |
```

Figure 6: Fallback Object Transfer - Machine to Machine flow

The steps in the diagram are as follows:

1. The client initiates a transfer request to the gaining registrar, providing the transfer authorization token that was obtained out-of-band from the losing registrar (e.g., via the registrant or a prior registrar-to-registrar agreement).
2. The gaining registrar submits the transfer request to the registry, including the opaque authorization token in the `RPP-Authorization` header.
3. The registry validates the authorization token against the losing registrar's records.
4. The registry executes the transfer and notifies the losing registrar.
5. The registry returns a successful transfer response to the gaining registrar.
6. The gaining registrar confirms the completed transfer to the client.

17. IANA Considerations

17.1. URN Sub-namespace for RPP (urn:ietf:params:rpp)

The IANA is requested to add the following value to the "IETF URN Sub-namespace for Registered Protocol Parameter Identifiers" registry, following the template in [RFC3553]: TODO: add filled in template, if we decide to use URN for profile identification, for example "urn:ietf:params:rpp:profile:example-profile"

```
Registry name: rpp
Specification: This Document
Repository: ?
Index value: ?
```

17.2. RPP registry group

The IANA is requested to create a new registry group for RPP, this will be used to group together all the RPP-related registries such as those for discovery URLs, extensions and profiles.

```
Name of the registry: RESTful Provisioning Protocol (RPP)
Reference: This Document
IANA Registry Reference: TODO
```

17.3. RPP Discovery registry

The IANA is requested to create a new registry for RPP discovery URLs, this registry will be used to register the well-known URLs for RPP discovery endpoints, used by RPP clients to discover the capabilities of a RPP server.

```
Name of the registry: RPP Discovery URLs
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `tld`: The top-level domain (TLD) for which the discovery URL is applicable, for example "example".
- `url`: The URL for the discovery endpoint, for example "https://rpp.example/.well-known/rpp".
- `description`: A human-readable description of the discovery URL and its intended use.

17.4. RPP Extension registry

The IANA is requested to create a new registry for RPP extensions, this registry will be used to register standardized extensions to the RPP protocol. Extensions are defined as additional features or capabilities that can be added to the base RPP protocol, for example support for additional resource types, additional operations or additional authentication methods.

```
Name of the registry: RPP Extensions
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `name`: The name of the extension, for example "RPP example extension".
- `version`: The version of the extension, for example "1.0".
- `url`: The URL for the extension specification, for example "https://www.iana.org/assignments/rpp-extensions/rpp-example-extension-1.0".
- `description`: A human-readable description of the extension and its intended use.

17.5. RPP Profile registry

The IANA is requested to create a new registry for RPP profiles, this registry will be used to register standardized profiles for the RPP protocol. Profiles are defined as specific configurations of the RPP protocol that are designed to meet the needs of specific use cases or environments, for example an EPP compatibility profile that defines a set of RPP features and behaviors that are compatible with the Extensible Provisioning Protocol (EPP) [RFC5730].

```
Name of the registry: RPP Profiles
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `name`: The name of the profile, for example "EPP compatibility profile".
- `id`: A unique URN identifier for the profile, for example "urn:ietf:params:rpp:profile:epp-compatibility-1.0".
- `version`: The version of the profile, for example "1.0".
- `url`: The URL for the profile specification, for example "https://www.iana.org/assignments/rpp-profiles/epp-compatibility-provisioning-profile-1.0".
- `description`: A human-readable description of the profile and its intended use.

17.6. RPP Result Codes Registry

The IANA is requested to create a new registry "RPP Result codes", this registry will be used to register RPP result codes defined in this document and in future RPP specifications and extensions.

```
Name of the registry: RPP Result codes
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

Fields to be registered:

- `code`: The RPP result code, for example "12000".
- `description`: A human-readable description of the result code and its intended use.

17.7. RPP Media Type (application/rpp+json)

The IANA is requested to add the following RPP media type to the "Media Types" registry, following the template in [RFC6838]:

```
Type name: application
Subtype name: rpp+json
Required parameters: version
Optional parameters: "N/A"
Encoding considerations: "N/A"
Security considerations: "N/A"
Interoperability considerations: "N/A"
Published specification: This document
Applications that use this media type: RPP protocol and extensions
Fragment identifier considerations: "N/A"
Additional information: "N/A"
Person & email address to contact for further information: Author's email address
Intended usage: COMMON
Restrictions on usage: "N/A"
Author: Document authors
Change controller: Document authors
Provisional registration: No
```

18. Internationalization Considerations

TODO

19. Security Considerations

RPP relies on the security of the underlying HTTP transport, hence the best common practices for securing HTTP described in [RFC9325] also apply to RPP and MUST be followed by RPP implementations.

Data confidentiality and integrity MUST be enforced. Every client and server interaction MUST be encrypted using TLS version 1.3 [RFC8446]. Future versions of TLS MAY be used as they become available and are deemed secure.

20. Change History

20.1. Version 05 to 06

- Expanded the "Authentication and Authorization" section to include OAuth 2.0 description. (Issue #33)

20.2. Version 04 to 05

- Added Boostrap and Discovery sections to the document, describing how a client can discover the location and capabilities of an RPP server
- Added IANA Considerations section with a request for new RPP discovery URLs, extensions and profile URLs.

20.3. Version 03 to 04

- Added a new section on versioning, describing how versioning is applied to different RPP elements. (Issue #39)
- Added IANA request for RPP Result codes registry, and added a table with example RPP result codes. (Issue #39)
- Added IANA request for URN sub-namespace for RPP. (Issue #39)
- Added a new "Profiles" section to describe how to define and use profiles. (Issue #43)
- Added a new section "Authentication and Authorization". (Issue #37)
- Updated the "Security Considerations" section to include transport security. (Issue #37)
- Added IANA registration request for the new RPP media type. (Issue #40)

20.4. Version 02 to 03

- Added use of Problem Detail [RFC9457] for error responses

20.5. Version 01 to 02

- Updated the examples, changed from "*.example.org" to "*.example"
- Merged the RPP-EPP-Code and RPP-Code headers into a single RPP-Code header
- Update the RPP-Authorization header to match the HTTP Authorization header format

- Added new process path segment and process representations
- Updated the Check request to now use an "availability" path segment and support both GET and HEAD methods

20.6. Version 00 to 01

- Updated "Request Headers" and "Response Headers" section
- Changed transfer resource URL and HTTP method for reject, approve and cancel, in order to make the API easier to use

20.7. Version 00 (draft-rpp-core) to 00 (draft-wullink-rpp-core)

- Renamed the document name to "draft-wullink-rpp-core"
- Removed sections: Design Considerations, Resource Naming Convention, Session Management, HTTP Layer, Content Negotiation, Object Filtering, Error Handling
- Renamed Commands section to Endpoints
- Removed text about extensions
- Changed naming to be less EPP like and more RDAP like

21. Acknowledgements

The authors would like to thank the following people for their helpful text contributions, comments and suggestions.

- Q Misell, AS207960 Cyfyngedig

22. References

22.1. Normative References

[I-D.kowalik-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-kowalik-rpp-data-objects-03, 2 March 2026, <https://datatracker.ietf.org/doc/html/draft-kowalik-rpp-data-objects-03>.

[REST] Fielding, R., "Architectural Styles and the Design of Network-based Software Architectures", 2000, <http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC2616] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1", RFC 2616, DOI 10.17487/RFC2616, June 1999, <https://www.rfc-editor.org/info/rfc2616>.

[RFC2782] Gulbrandsen, A., Vixie, P., and L. Esibov, "A DNS RR for specifying the location of services (DNS SRV)", RFC 2782, DOI 10.17487/RFC2782, February 2000, <https://www.rfc-editor.org/info/rfc2782>.

[RFC3553] Mealling, M., Masinter, L., Hardie, T., and G. Klyne, "An IETF URN Sub-namespace for Registered Protocol Parameters", BCP 73, RFC 3553, DOI 10.17487/RFC3553, June 2003, <https://www.rfc-editor.org/info/rfc3553>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/info/rfc3986>.

[RFC5730] Hollenbeck, S., "Extensible Provisioning Protocol (EPP)", STD 69, RFC 5730, DOI 10.17487/RFC5730, August 2009, <https://www.rfc-editor.org/info/rfc5730>.

[RFC5731] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Domain Name Mapping", STD 69, RFC 5731, DOI 10.17487/RFC5731, August 2009, <https://www.rfc-editor.org/info/rfc5731>.

[RFC5732] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Host Mapping", STD 69, RFC 5732, DOI 10.17487/RFC5732, August 2009, <https://www.rfc-editor.org/info/rfc5732>.

[RFC5733] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Contact Mapping", STD 69, RFC 5733, DOI 10.17487/RFC5733, August 2009, <https://www.rfc-editor.org/info/rfc5733>.

[RFC6570] Gregorio, J., Fielding, R., Hadley, M., Nottingham, M., and D. Orchard, "URI Template", RFC 6570, DOI 10.17487/RFC6570, March 2012, <https://www.rfc-editor.org/info/rfc6570>.

[RFC6648] Saint-Andre, P., Crocker, D., and M. Nottingham, "Deprecating the "X-" Prefix and Similar Constructs in Application Protocols", BCP 178, RFC 6648, DOI 10.17487/RFC6648, June 2012, <https://www.rfc-editor.org/info/rfc6648>.

[RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <https://www.rfc-editor.org/info/rfc6749>.

[RFC6838] Freed, N., Klensin, J., and T. Hansen, "Media Type Specifications and Registration Procedures", BCP 13, RFC 6838, DOI 10.17487/RFC6838, January 2013, <https://www.rfc-editor.org/info/rfc6838>.

[RFC7519] Jones, M., Bradley, J., and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, DOI 10.17487/RFC7519, May 2015, <https://www.rfc-editor.org/info/rfc7519>.

[RFC7617] Reschke, J., "The 'Basic' HTTP Authentication Scheme", RFC 7617, DOI 10.17487/RFC7617, September 2015, <https://www.rfc-editor.org/info/rfc7617>.

[RFC8414] Jones, M., Sakimura, N., and J. Bradley, "OAuth 2.0 Authorization Server Metadata", RFC 8414, DOI 10.17487/RFC8414, June 2018, <https://www.rfc-editor.org/info/rfc8414>.

[RFC8446] Rescorla, E., "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, DOI 10.17487/RFC8446, August 2018, <https://www.rfc-editor.org/info/rfc8446>.

[RFC8725] Sheffer, Y., Hardt, D., and M. Jones, "JSON Web Token Best Current Practices", BCP 225, RFC 8725, DOI 10.17487/RFC8725, February 2020, <https://www.rfc-editor.org/info/rfc8725>.

[RFC8941] Nottingham, M. and P. Kamp, "Structured Field Values for HTTP", RFC 8941, DOI 10.17487/RFC8941, February 2021, <https://www.rfc-editor.org/info/rfc8941>.

[RFC9068] Bertocci, V., "JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens", RFC 9068, DOI 10.17487/RFC9068, October 2021, <https://www.rfc-editor.org/info/rfc9068>.

[RFC9110] Fielding, R., Ed., Nottingham, M., Ed., and J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, DOI 10.17487/RFC9110, June 2022, <https://www.rfc-editor.org/info/rfc9110>.

[RFC9205] Nottingham, M., "Building Protocols with HTTP", BCP 56, RFC 9205, DOI 10.17487/RFC9205, June 2022, <https://www.rfc-editor.org/info/rfc9205>.

[RFC9325] Sheffer, Y., Saint-Andre, P., and T. Fossati, "Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)", BCP 195, RFC 9325, DOI 10.17487/RFC9325, November 2022, <https://www.rfc-editor.org/info/rfc9325>.

[RFC9396] Lodderstedt, T., Richer, J., and B. Campbell, "OAuth 2.0 Rich Authorization Requests", RFC 9396, DOI 10.17487/RFC9396, May 2023, <https://www.rfc-editor.org/info/rfc9396>.

[RFC9457] Nottingham, M., Wilde, E., and S. Dalal, "Problem Details for HTTP APIs", RFC 9457, DOI 10.17487/RFC9457, July 2023, <https://www.rfc-editor.org/info/rfc9457>.

[RFC9535] Gössner, S., Ed., Normington, G., Ed., and C. Bormann, Ed., "JSONPath: Query Expressions for JSON", RFC 9535, DOI 10.17487/RFC9535, February 2024, <https://www.rfc-editor.org/info/rfc9535>.

[SemVer] Semantic Versioning, "Semantic Versioning 2.0.0", <https://semver.org/>.

22.2. Informative References

[RFC8748] Carney, R., Brown, G., and J. Frakes, "Registry Fee Extension for the Extensible Provisioning Protocol (EPP)", RFC 8748, DOI 10.17487/RFC8748, March 2020, <https://www.rfc-editor.org/info/rfc8748>.

Authors' Addresses

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/